Mouse RNA-Seq — FastQC (raw vs trimmed)

BIOL550 — Weekly Report — Mar 4, 2026

What I accomplished since the previous report

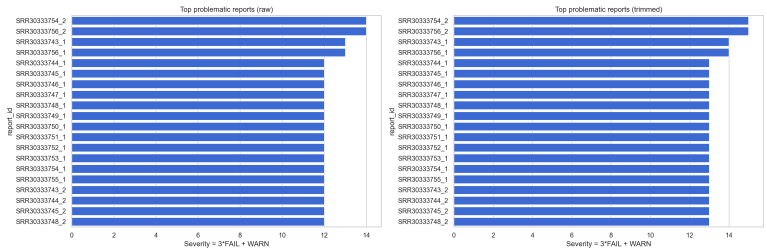
This week, in collaboration with my team, I moved our shared pipeline to the mouse RNA-Seq dataset (BioProject `PRJNA1017789`; GEO Series `GSE243308`) and repeated the full workflow from scratch. This directly follows last week's stated goals: re-running FastQC on trimmed reads, repeating SRR-level QC analysis, and confirming the dataset's paper and experimental design. After realizing our earlier candidate (zebrafish) dataset was single-cell RNA-Seq and not suitable for this workflow, we pivoted to this bulk mouse project, downloaded 26 paired-end runs (`SRR30333743`–`SRR30333768`) using SRA Toolkit, and generated raw and trimmed FastQC reports for all runs.

Also, following our zebrafish project's approach, I wrapped the steps into a resumable pipeline (download → FastQC → FASTX trim → FastQC) so the team can run consistently without overloading shared compute/storage resources. As a result, we produced **52 raw** FastQC reports and **52 trimmed** FastQC reports (26 SRRs × 2 mates in each stage), along with summary tables and figures for interpretation.

Quick Review of FastQC Output Findings

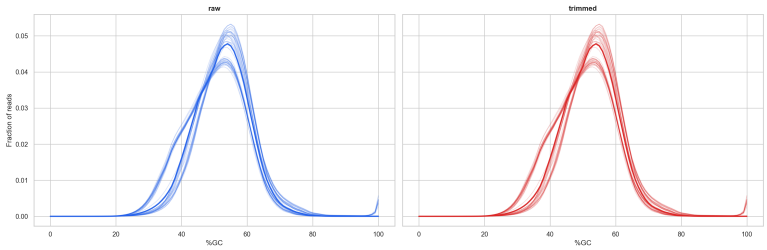
To summarize FastQC across the dataset, I used Python and pandas to parse each `*_fastqc.zip` and calculate a per-report severity score (`3×FAIL + WARN`). This let us quickly identify which reports (and SRRs) concentrate the most WARN/FAIL modules, which modules are consistently problematic across the project, and how those patterns compare between raw and trimmed QC. Following the same approach as the previous report, below I show the top problematic reports and a module-level summary across all FastQC outputs, with trimmed QC included as an additional comparison.

FastQC overview (raw vs trimmed): module outcomes (right) + most problematic reports (left)



module	raw (n=52)		trimmed (n=52)		Δ (trimmed − raw)	
	fail	warn	fail	warn	fail	warn
Adapter Content	52	0	52	0	0	0
Sequence Duplication Levels	52	0	52	0	0	0
Per base sequence content	26	26	26	26	0	0
Per sequence GC content	0	35	0	35	0	0
Basic Statistics	0	0	0	0	0	0
Overrepresented sequences	2	13	2	13	0	0
Per base N content	0	0	0	0	0	0
Per base sequence quality	0	0	0	0	0	0
Per sequence quality scores	0	0	0	0	0	0
Per tile sequence quality	28	0	28	0	0	0
Sequence Length Distribution	0	0	0	52	0	52

As in the previous report, the left panel ranks individual FastQC reports by severity (`3×FAIL + WARN`) to prioritize which SRRs to inspect first. The right table summarizes module PASS/WARN/FAIL counts across all reports to distinguish "global" patterns. Here, we also include trimmed QC (trimmed columns) to compare modules before vs after trimming: `Adapter Content` fails across all reports in both stages, and `Sequence Length Distribution` flips from PASS (raw) to WARN (trimmed) because trimming introduces variable read lengths. Overall, trimming improves tail quality but does not remove adapter signal (it trims low-quality tails, not technical sequence), so adapter clipping (e.g., FASTX `fastx_clipper`) is required.

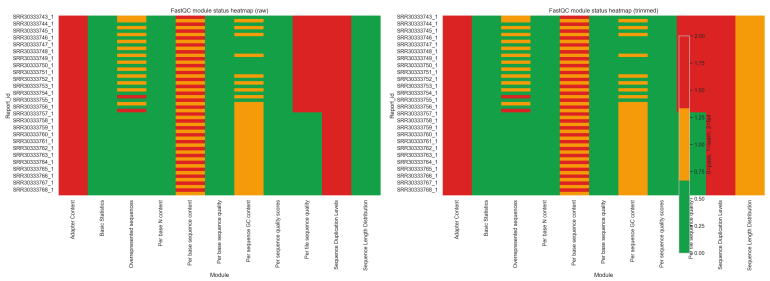


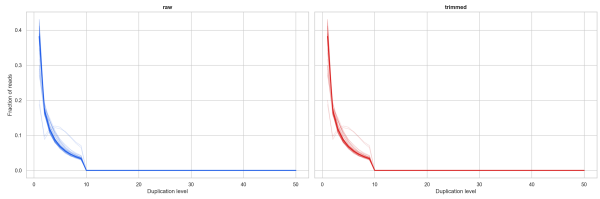
Per-sequence GC content (raw vs trimmed)

GC profiles are similar pre- vs post-trim (peaks align; no strong secondary mode), which argues against an obvious contamination signal at the dataset level. The module remains frequently WARN (`35/52` in both stages), consistent with RNA-Seq composition effects rather than a single outlier batch.

FastQC module status by report (raw vs trimmed)

The same dataset-wide flags dominate in both stages (`Adapter Content`, `Sequence Duplication Levels`, and many runs split WARN/FAIL for `Per base sequence content`). The most systematic stage change is `Sequence Length Distribution` switching to WARN after trimming, which confirms trimming is active but does not resolve adapter signal; this points to adding adapter clipping (`fastx_clipper`) before repeating FastQC.



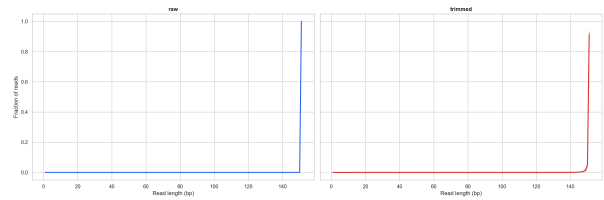


Sequence duplication levels (raw vs trimmed)

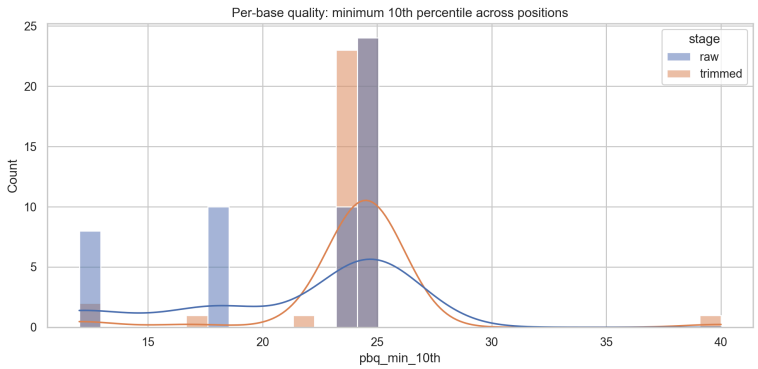
Duplication remains high in both stages (FAIL across all reports), which is common in RNA-Seq due to highly expressed transcripts. Trimming does not change this pattern, so we treat duplication as a dataset-wide characteristic unless extreme outliers appear.

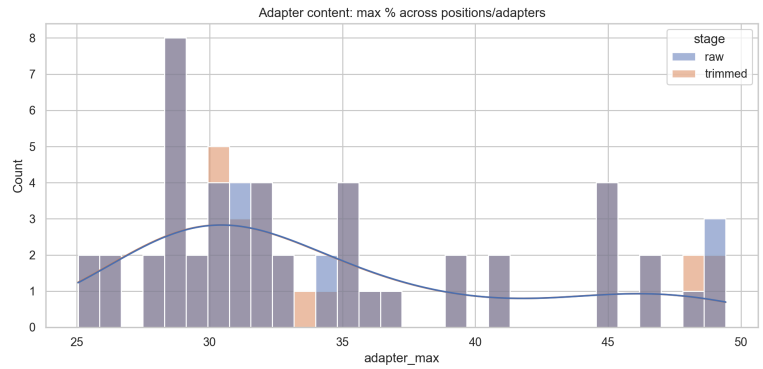
Sequence length distribution (raw vs trimmed)

Raw reads are fixed-length (151 bp), while trimmed reads show a spread of shorter lengths (hence WARN for `Sequence Length Distribution` after trimming). This is an expected side effect and contributes to small increases in the simple severity score; we will confirm we are not over-trimming into overly short reads for alignment.



Key summary metrics (raw vs trimmed): per-base quality + adapter signal





Quality trimming shifts tail-quality upward, but adapter signal is unchanged between raw and trimmed; this supports adding an explicit adapter clipping step before repeating FastQC and re-checking `Adapter Content` / `Overrepresented sequences`.

Methods used (commands + parameters)

We used the same toolchain and workflow as the previous report: SRA Toolkit to download paired-end FASTQs, FastQC to assess read quality (raw and trimmed), and FASTX for quality trimming. For this mouse dataset, quality trimming used `fastq_quality_trimmer` with `-Q33 -t 20 -l 30` (Phred+33, trim bases <Q20, keep reads ≥30 bp). We then summarized FastQC by parsing `*_fastqc.zip` and comparing raw vs trimmed at the module + SRR level.

Manual Commands for Each SRR

```
fastq-dump --split-files --gzip SRR30333743 # download paired FASTQs
fastqc -t 1 SRR30333743_1.fastq.gz SRR30333743_2.fastq.gz # QC both mates
fastq_quality_trimmer -Q33 -t 20 -l 30 # FASTX: trim low-Q 3' tails (stdin->stdout)
```

Note: To automate the manual MultiQC validation step (after verifying a few SRRs by hand), I used a small crawler to iterate over the local FastQC HTML reports to have each report as a parseable object, then used pandas to convert each module table into DataFrames for consistent aggregation. Finally, I used pandas plotting plus lightweight HTML/CSS templating from prior reports to generate the summary figures and highlighted tables, speeding review while preserving accuracy as we restarted from zero.

Problems encountered

First, our earlier candidate dataset turned out to be **single-cell RNA-Seq**, which does not match this bulk-style download→FastQC→FASTX workflow; we pivoted to the mouse bulk RNA-seq dataset and re-ran the entire pipeline. Second, the SRRs are large and the server is shared, so the pipeline had to be resumable and concurrency-limited. To speed up the process while addressing this, we ran a parallelized end-to-end pipeline with clear progress monitoring. Lastly, the dominant FastQC flags (adapter signal and high duplication) are not resolved by quality trimming alone, so we need to add adapter clipping and re-evaluate before downstream analysis.

Goals for the coming week

Next week, in collaboration with my team, I will (1) add adapter clipping (FASTX `fastx_clipper`) and quantify whether `Adapter Content` and `Overrepresented sequences` improve after clipping, (2) re-run FastQC after clipping and repeat the same SRR/module comparisons to confirm improvements without excessive read shortening, (3) generate a consolidated QC summary (e.g., MultiQC) for the finalized raw/trimmed/clipped outputs, and (4) integrate the confirmed experimental design/metadata for `PRJNA1017789`/`GSE243308` into our analysis plan (grouping samples and specifying contrasts) so we can proceed into alignment/quantification.